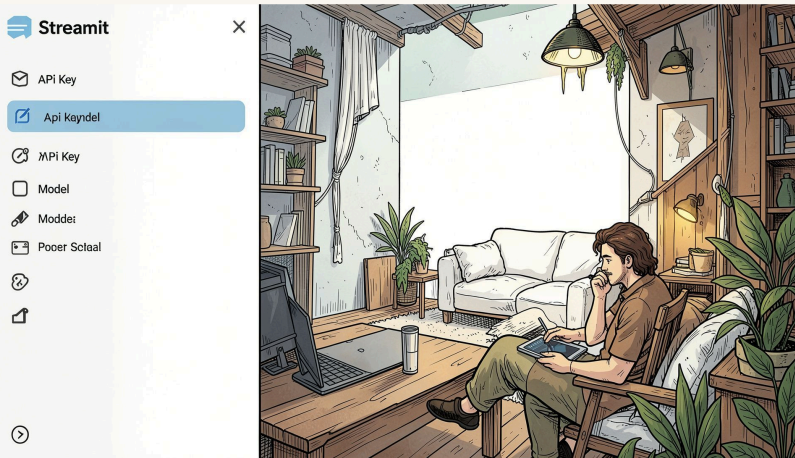




AI Automation Chatbot — Quick product brief

Streamlit-hosted assistant focused on AI automation queries: fast, deterministic, and narrowly scoped to automation topics (Zapier, n8n, Make, orchestration patterns, integration design).

OVERVIEW



What it is

This chatbot accepts user prompts via a Streamlit UI and returns answers limited to AI automation topics. It enforces scope-filtering so off-topic requests are ignored, improving signal quality for product and developer users evaluating RAG-driven document QA tools.

□ Theme accents: use #D3C5B6 for UI elements and accents in the app

Key features



Focused scope

Answers restricted to AI automation topics — higher relevance, lower hallucination risk for automation guidance.



Smart filtering

Relevance classifier drops or politely rejects unrelated queries, improving user trust and moderation.

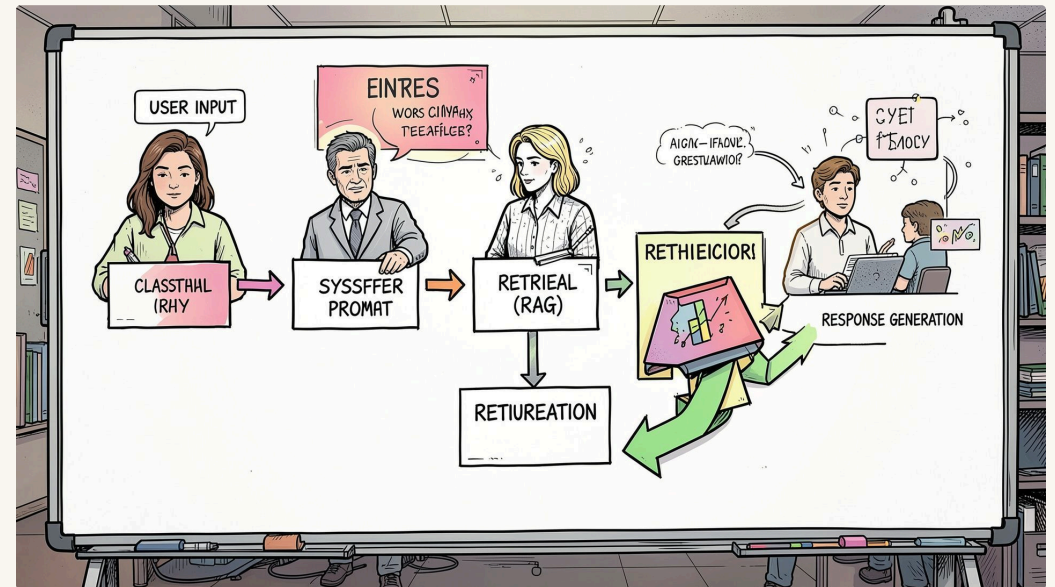


Integration guidance

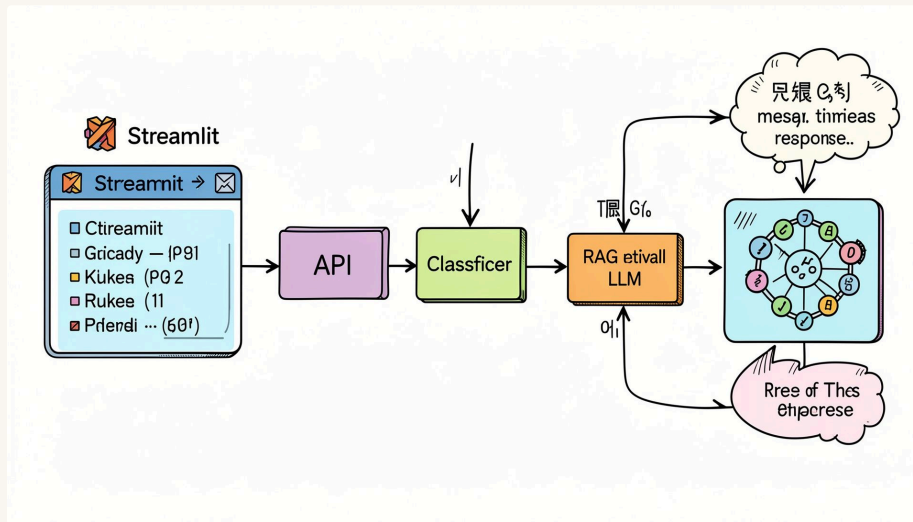
Practical examples for Zapier, Make, n8n and orchestration patterns, plus code snippets and configuration tips.

How it works — high level

1) User submits query via Streamlit UI. 2) System prompt constrains assistant to automation topics. 3) Relevance filter classifies request; non-automation queries are ignored. 4) If relevant, the model responds with concise, actionable guidance and optional code/config examples.



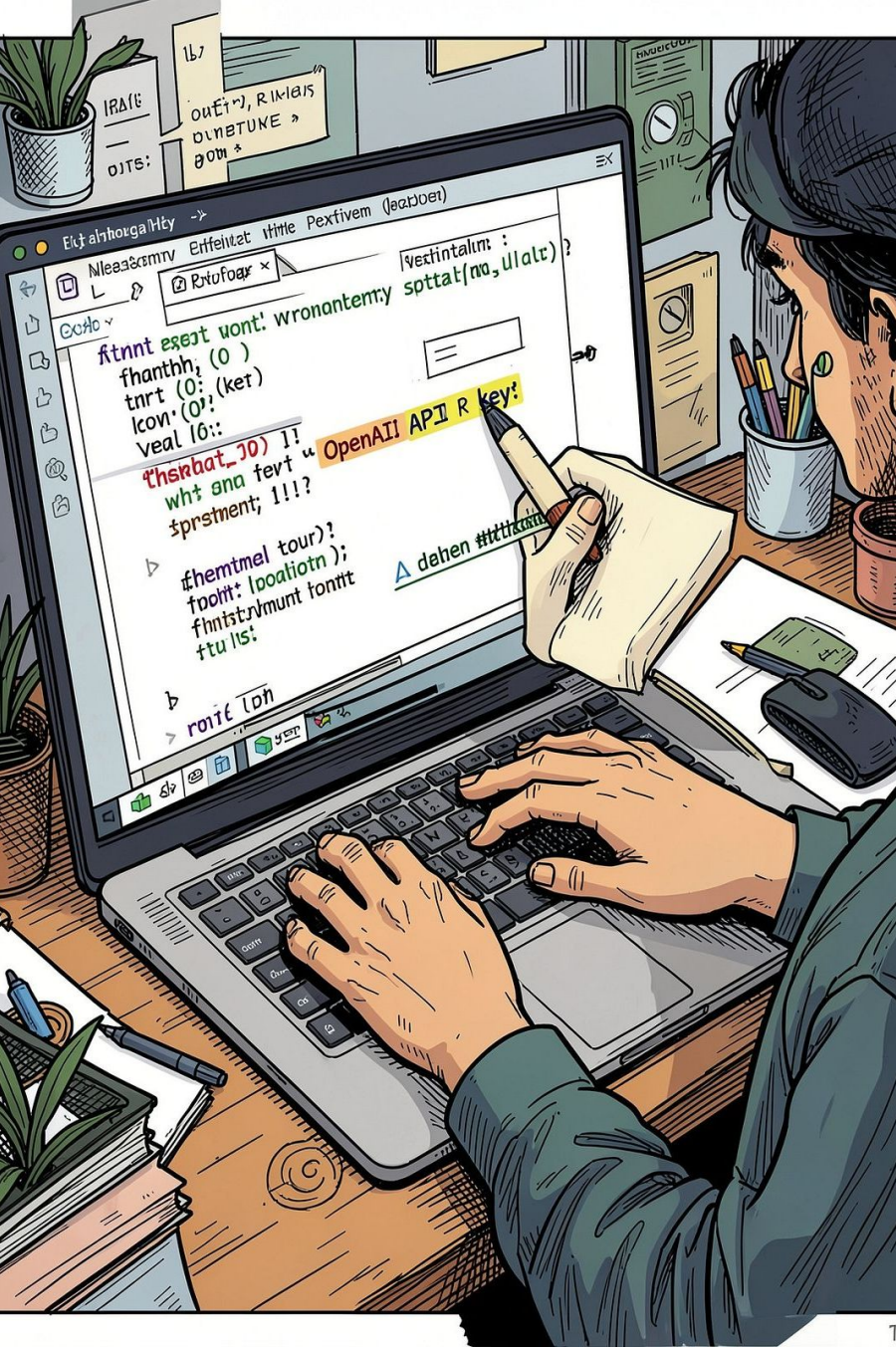
ARCHITECTURE



Components

- Streamlit frontend: API key input, model selection, clear chat
- Relevance classifier: lightweight model or rules to enforce scope
- RAG retrieval (optional): domain docs, automation guides, integration playbooks
- LLM backend: system prompt enforces automation-only responses

Design goal: predictable, auditable answers with traceable sources when using RAG.



Installation & configuration

Clone repository, install Python dependencies, set environment variables, and run Streamlit. Minimal steps provided for developer evaluation:

1. `git clone https://github.com/your-username/ai-automation-chatbot.git`
2. `cd ai-automation-chatbot && pip install -r requirements.txt`
3. `export OPENAI_API_KEY="your_api_key_here"`
4. `streamlit run app.py`



Sample prompts & responses

Responses prioritise practical steps, code snippets, and configuration hints.

Prompt

“How do I trigger a Slack message from a new Airtable record?”

Answer

Explain connector steps: Airtable webhook or poll, transform mapping, Slack API call with auth, example payload and error handling notes.



Limitations & safety

To reduce risk and scope creep the assistant:

- Rejects or ignores non-automation queries
- Avoids speculative architecture advice outside integrations domain
- Does not provide privileged credentials or sensitive action guidance

⚠ Consider a moderation layer and logging for auditability in production

Roadmap & improvements

Short term

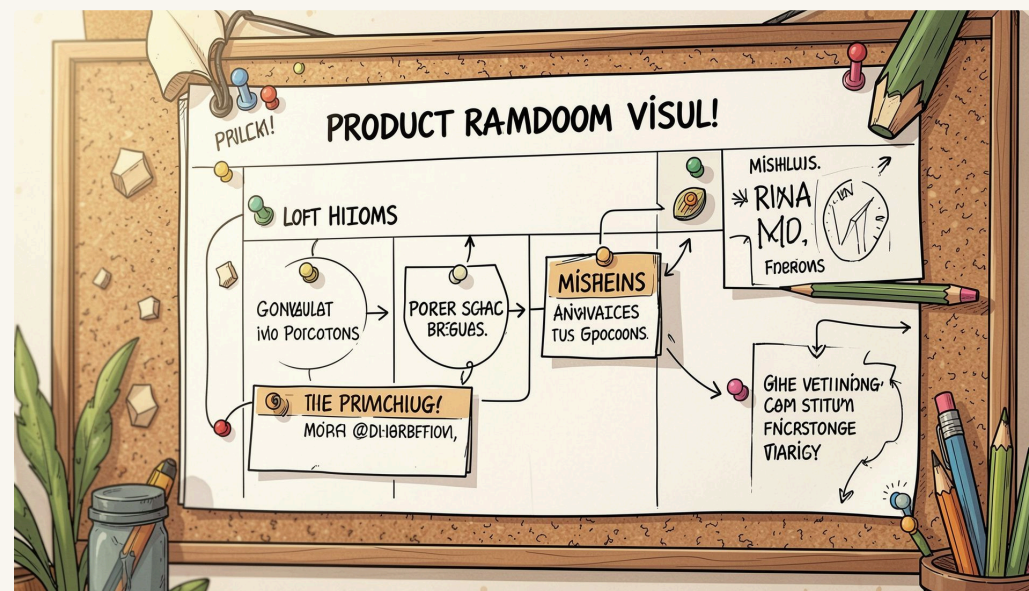
Improve classifier precision, add example library for Zapier/Make templates.

Medium term

Introduce RAG retrieval from internal playbooks with source citations.

Long term

Multi-language support, voice I/O, and secure connector credential vaulting.





Next steps for evaluators

For developer and PM review: run the app locally, test a matrix of relevant prompts, evaluate classifier false positive/negative rates, and experiment with RAG retrieval for source-backed answers. Share feedback and propose priority items from the roadmap.

📄 Streamlit link: <https://aiautomationchatbot-1abdullahh.streamlit.app/>